06

Creando tu primer proyecto



El asistente de creación de Astro, plantillas y opciones

CONCEPTO TEMA 6

Un solo comando para empezar

Terminal

```
$ npm create astro@latest

# El asistente te pregunta (en inglés):
  ¿Dónde crear el proyecto?  ./mi-primer-astro
  ¿Cómo quieres empezar?     Plantilla mínima
  ¿Instalar dependencias?    Sí
  ¿Iniciar un repositorio?   Sí
✔  ¡Proyecto listo!

$ cd mi-primer-astro
$ npm run dev
```

- `npm create astro@latest` descarga y ejecuta el **asistente oficial**.
- `@latest` asegura que uses la **última versión**.
- Las preguntas exactas pueden variar un poco entre versiones del asistente.
- También funcionan `pnpm create astro@latest` y `yarn create astro`.

CONCEPTO TEMA 6

Qué significa cada pregunta del asistente

1



Carpeta

Dónde se creará el proyecto. Usa un nombre sin espacios: `mi-primer-astro`.

2



Plantilla

Proyecto de inicio: **básico**, **mínimo** (vacío) u otro. Hoy usaremos el **mínimo**.

3



Dependencias

Responde **sí** para que ejecute `npm install` automáticamente.

4



Git

Responde **sí** para crear un repositorio local listo para GitHub.

TÉRMINO NUEVO TEMA 6

Plantilla y dependencia

Plantilla (template)

Proyecto **de inicio** ya armado que copias para no empezar de cero. Astro tiene plantillas oficiales y de la comunidad.

Ejemplos

`minimal` (vacío), `basics`, `blog`, `portfolio`.

Dependencia

Paquete externo que tu proyecto necesita para funcionar. Se lista en `package.json` y se descarga con `npm install`.

Ejemplo

`astro` es la dependencia principal de tu proyecto.

TÉRMINO NUEVO TEMA 6

package.json y node_modules

package.json

Archivo que **describe tu proyecto**: nombre, dependencias y **scripts** (comandos como `dev` o `build`).

Analogía
La lista de ingredientes y pasos de una receta.

node_modules/

Carpeta donde npm **descarga** el código de todas las dependencias. Es muy pesada y **no se sube a GitHub**.

Dato
Si la borras, la recuperas con `npm install`.

CONCEPTO TEMA 6

Opciones útiles al crear un proyecto

Terminal

```
# Usar una plantilla oficial
$ npm create astro@latest -- --template minimal

# Usar un repositorio de GitHub
$ npm create astro@latest -- --template usuario/repo

# Crear e instalar una integración
$ npm create astro@latest -- --add react
```

- El `--` separa las opciones de **npm** de las del **asistente** de Astro.
- `--template` acepta una plantilla oficial o un repositorio `usuario/repo`.
- `--add` instala integraciones en el mismo paso.

EJEMPLO RESUELTO TEMA 6

Creo y abro mi primer proyecto

Enunciado: Crea el proyecto `mi-primer-astro` con la plantilla mínima, ábrelo en VS Code e inicia el servidor de desarrollo.

Terminal

```
$ npm create astro@latest mi-primer-astro
  # plantilla: mínima (minimal)
  # dependencias y git: Sí
$ cd mi-primer-astro
$ code .        # abre la carpeta en VS Code
$ npm run dev
  Local  http://localhost:4321/
```

1. El nombre de la carpeta puede ir **en el mismo comando**.
2. `cd` entra a la carpeta del proyecto.
3. `code .` abre la carpeta actual en VS Code.
4. `npm run dev` inicia el servidor de desarrollo.

Resultado: Abre `http://localhost:4321` en el navegador: verás tu página de inicio.

EJERCICIOS SUGERIDOS TEMA 6

Practica lo aprendido

1. **Dos plantillas.** Crea un proyecto con `--template minimal` y otro con `--template basics`. Compara qué archivos trae cada uno.

2. **Plantilla de blog.** Crea un proyecto con `--template blog`, ejecútalo y navega por sus páginas. Anota 3 cosas que te llamen la atención.

3. **Borra y recupera.** En una copia de tu proyecto, borra `node_modules/` e intenta `npm run dev`. Luego ejecuta `npm install` y vuelve a probar.

07

La estructura de un proyecto Astro



Qué hay en cada carpeta y qué reglas debes respetar

CONCEPTO TEMA 7

El mapa de tu proyecto

```
mi-primer-astro/
├── public/              ← sin procesar
│   └── favicon.svg
├── src/                 ← código fuente
│   ├── components/      ← piezas reutilizables
│   ├── layouts/         ← estructuras compartidas
│   ├── pages/           ← OBLIGATORIA: rutas
│   │   └── index.astro
│   └── styles/          ← CSS global
├── astro.config.mjs     ← configuración
├── package.json         ← dependencias y scripts
└── tsconfig.json        ← ayuda al editor
```

- Solo `src/pages/` es **obligatoria** y **reservada** por Astro.
- Las demás carpetas son **convenciones**: puedes organizarlas a tu gusto.
- La plantilla mínima trae solo lo esencial; tú creas el resto.

CONCEPTO TEMA 7

Las carpetas de src/

src/pages/

Obligatoria. Cada archivo es una **ruta** (URL) de tu sitio. Sin ella, tu sitio no tiene páginas.

src/components/

Componentes reutilizables: `.astro` o de frameworks como React (`.jsx`).

src/layouts/

Estructuras que comparten varias páginas: `<head>`, encabezado, pie.

src/styles/

Archivos CSS o Sass globales. Es una convención, no un requisito.

CONCEPTO TEMA 7

src/ vs. public/

src/ · se procesa

- Astro **procesa, optimiza y empaqueta** estos archivos
- Aquí va **tu CSS y tu JS**
- Las imágenes que **importas** se pueden optimizar
- Ej.: componentes, páginas, estilos

public/ · se copia tal cual

- Se copia **sin cambios** a la carpeta final
- No se optimiza ni se empaqueta
- Ideal para archivos que no deben cambiar de nombre
- Ej.: `favicon.svg`, `robots.txt`, un PDF

Un archivo `public/cv.pdf` se abre en `http://localhost:4321/cv.pdf` (sin "public" en la URL).

CONCEPTO TEMA 7

Los archivos de configuración

astro.config.mjs

astro.config.mjs

```mjs
import { defineConfig } from "astro/config";

export default defineConfig({
  // integraciones, adaptador,
  // i18n, fuentes, etc.
});
```

Scripts en package.json

package.json

```json
{
  "scripts": {
    "dev": "astro dev",
    "build": "astro build",
    "preview": "astro preview",
    "astro": "astro"
  }
}
```

`npm run dev` ejecuta el script **dev** → `astro dev`. Así funcionan los tres comandos principales.

TÉRMINO NUEVO TEMA 7

Ruta y archivo estático

Ruta

Dirección (URL) a la que responde una página. En Astro se crea automáticamente a partir del nombre del archivo en `src/pages/`.

Ejemplo

`src/pages/contacto.astro` → `/contacto`.

Archivo estático (asset)

Archivo que se entrega **tal cual**, sin generarse ni modificarse: imágenes, fuentes, PDF, íconos.

Ejemplo

`public/robots.txt` → `/robots.txt`.

EJEMPLO RESUELTO TEMA 7

Mi página "Sobre mí"

Enunciado: Crea la página `/sobre-mi` con tu nombre en una variable y un enlace de regreso al inicio.

src/pages/sobre-mi.astro

```
---
const nombre = "Ana";
---
<html lang="es">
  <head><title>Sobre mí</title></head>
  <body>
    <h1>Hola, soy {nombre}</h1>
    <a href="/">Volver al inicio</a>
  </body>
</html>
```

1. El archivo `sobre-mi.astro` en `src/pages/` crea la ruta `/sobre-mi`.
2. La expresión `{nombre}` inserta el valor de la variable.
3. `index.astro` siempre corresponde a `/`.

Resultado: Visita `http://localhost:4321/sobre-mi`: la página aparece sin reiniciar nada.

EJERCICIOS SUGERIDOS TEMA 7

Practica lo aprendido

1. **Más páginas.** Crea `contacto.astro` y `blog/index.astro`. ¿Qué URL tiene cada una? Enlázalas desde el inicio.

2. **Un PDF público.** Coloca un PDF en `public/` y enlázalo desde la página "Sobre mí" con `<a href="/archivo.pdf">`.

3. **Experimento.** En una copia del proyecto, renombra `src/pages/` a `src/paginas/`. ¿Qué pasa al visitar `/`? Explica por qué.

08

Desarrollar, construir y previsualizar



Los comandos dev, build y preview, y las opciones del CLI

CONCEPTO TEMA 8

Los 3 comandos principales

Comando	Qué hace	Cuándo usarlo	Resultado
`npm run dev`	Inicia el servidor de desarrollo con recarga automática	Mientras programas	`http://localhost:4321`
`npm run build`	Construye la versión final del sitio	Antes de publicar	Carpeta `dist/`
`npm run preview`	Sirve localmente lo que generó el build	Para revisar antes de publicar	`http://localhost:4321`

Para detener cualquiera de los servidores presiona `Ctrl + C` en la terminal.

CONCEPTO TEMA 8

npm run dev en acción

Terminal

```
$ npm run dev

  astro  v7.3.x  ready

  | Local    http://localhost:4321/
  | Network  use --host to expose

# Editas src/pages/index.astro…
# …y el navegador se actualiza solo
```

- Astro **observa** los cambios en `src/` y actualiza la vista al instante.
- **No** necesitas reiniciar el servidor al editar.
- La salida exacta puede variar ligeramente según la versión.

TÉRMINO NUEVO TEMA 8

localhost y HMR

localhost y puerto

localhost significa "esta misma computadora". El **puerto** es un número que identifica qué programa responde. Astro usa el **4321**.

Ejemplo

```
http://localhost:4321/sobre-mi
```

HMR (Hot Module Replacement)

Recarga en caliente: reemplaza solo la parte que cambiaste sin recargar todo ni perder lo que estabas viendo.

Analogía

Cambiar una llanta sin bajar a los pasajeros del auto.

CONCEPTO TEMA 8

Construir y previsualizar

Terminal

```
$ npm run build
  ▶ src/pages/index.astro
  ▶ src/pages/sobre-mi.astro
✓ Completed — archivos en dist/

$ npm run preview
  ┃ Local    http://localhost:4321/

# preview muestra el sitio TAL COMO
# quedó en el último build
```

- `build` genera la **versión de producción** en `dist/`.
- Si el código tiene errores (por ejemplo, una etiqueta sin cerrar), el build **falla** y te avisa.
- Si editas algo, debes **volver a hacer build** para verlo en `preview`.

TÉRMINO NUEVO TEMA 8

Build y la carpeta dist/

Definición

Build es el proceso de **construir** la versión final del sitio. El resultado se guarda en `dist/` (de *distribution*): HTML, CSS, JS e imágenes listos para publicar.

Importante

Nunca edites `dist/` a mano: se **borra y regenera** en cada build. Tampoco se sube a GitHub.



CONCEPTO TEMA 8

Opciones y comandos útiles del CLI

Comando u opción	Para qué sirve
`npm run dev -- --port 3000`	Usar otro puerto en lugar del 4321
`npm run dev -- --open`	Abrir el navegador automáticamente
`npm run dev -- --host`	Ver el sitio desde tu celular en la misma red Wi-Fi
`npx astro add <integración>`	Agregar integraciones (Tailwind, React, adaptadores…)
`npx astro info`	Ver versiones de Astro, Node y del sistema (útil para pedir ayuda)
`npx astro docs`	Abrir la documentación oficial desde la terminal

Con `npm run` debes poner `--` antes de las opciones para que lleguen a Astro.

CONCEPTO TEMA 8

Novedad de Astro 7: servidor en segundo plano

Terminal

```
$ npx astro dev --background
  Dev server running at http://localhost:4321

$ npx astro dev status    # ¿está corriendo?
$ npx astro dev logs      # ver sus mensajes
$ npx astro dev stop      # detenerlo
```

- Ejecuta el servidor **sin ocupar la terminal**.
- Pensado para **agentes de IA** y automatizaciones; Astro detecta a los agentes y lo activa solo.
- Si intentas iniciar otro, te devuelve el que ya está corriendo.
- Para la clase usaremos `npm run dev` normal.

EJEMPLO RESUELTO TEMA 8

Verifico que mi sitio envía 0 KB de JS

Enunciado: Construye el proyecto y comprueba que la página "Sobre mí" no envía JavaScript al navegador.

Terminal

```
$ npm run build
✓ Completed
$ npm run preview
# En el navegador:
#   1. Abre /sobre-mi
#   2. F12 → Red (Network) → filtro JS
#   3. Recarga la página (F5)
0 archivos JS descargados
```

1. `build` genera `dist/sobre-mi/index.html`.
2. `preview` sirve ese HTML como lo haría el hosting.
3. En **Red/Network**, filtra por **JS** y recarga.
4. Lista vacía: el `{nombre}` se resolvió **al construir**, no en el navegador.

Resultado: La página no descarga ningún archivo JavaScript: **cero JS por defecto**, comprobado.

EJERCICIOS SUGERIDOS TEMA 8

Practica lo aprendido

1. **Otro puerto.** Inicia el servidor en el puerto 3000 con apertura automática del navegador.

2. **Explora dist/.** Ejecuta `npm run build` y revisa `dist/`. ¿Cuántos `.html` hay? ¿Hay algún `.js`?

3. **Preview desactualizado.** Con `preview` corriendo, cambia el texto de una página. ¿Se ve el cambio? ¿Qué debes hacer para verlo?

4. **Pide ayuda como un pro.** Ejecuta `npx astro info` y copia el resultado: es lo que se pega al reportar un problema.